LoRA: Low-Rank Adaptation of Large Language Models — A Reduced-Scale Reproduction and Ablation Study

Manya Dixit
Student ID: 24CS2026
Programme: IDD
Assignment DL-051

Abstract—Hu et al. (ICLR 2022) show that the update to a pretrained weight matrix can be well approximated by a low-rank matrix, allowing large language models to be adapted to downstream tasks with orders of magnitude fewer trainable parameters (LoRA). This work implements the core contribution of the paper from scratch in PyTorch—a custom LoRALinear layer, without any adapter library—and integrates it into a pretrained DistilBERT for GLUE SST-2 sentiment classification. We compare full fine-tuning against LoRA at ranks $r \in \{2, 4, 8\}$ under identical data, evaluation, and training settings, and ablate the rank. On a fixed training subset of 8,000 examples (2 epochs, seed 42), full fine-tuning trains 66,955,010 of the model's 66,955,010 parameters (100.000%%) versus only 665,858 parameters (0.993%%) for LoRA r=4, while reaching 88.65% (full FT) versus 87.84% (LoRA r=4) SST-2 validation accuracy. Peak measured device memory and training time are reported from the same runs. An identical re-run on a Google Colab T4 GPU confirms every conclusion (accuracies within 1.2 points, with exact CUDA memory accounting). We carefully distinguish results reported in the original paper (RoBERTa/GPT-3 scale) from our reduced-scale adaptation, and we state the limitations of a single-seed, subset, CPU/GPU-local study.

I. Introduction and Motivation

Pretraining large Transformers on generic corpora and fine-tuning them on downstream tasks is the dominant paradigm in NLP [5]. Full fine-tuning, however, updates every pretrained parameter and, for each task, produces a separate copy of the entire model. For a 175-billion-parameter model this means hundreds of billions of task-specific numbers to store and distribute, and it requires optimizer states for all parameters during training. In the original paper's accounting, training GPT-3 175B with Adam consumes about 1.2 TB of GPU memory, of which roughly two thirds is optimizer and gradient state for the parameters being updated [1].

The observation behind LoRA [1] is that although a fine-tuned model has a full-size weight matrix $W = W_0 + \Delta W$, the change ΔW produced by fine-tuning has low intrinsic rank: it can be well approximated by $\Delta W = BA$ with $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$ and $r \ll \min(d, k)$. Keeping W_0 frozen and learning only A and B reduces the number of trainable parameters per adapted layer from $d \cdot k$ to $r(d + k)$—for the query/value projections of DistilBERT ($d = k = 768$) this is a reduction from 590,592 to 3,072 parameters already at rank r=2.

Contributions of this assignment. (1) A clean, tested, from-scratch PyTorch implementation of LoRA (Section III); (2) a controlled comparison of full fine-tuning versus LoRA ($r = 2, 4, 8$) on DistilBERT + SST-2 with identical data and evaluation (Sections IV and V); (3) a rank ablation with measured accuracy, parameter, memory, and wall-clock costs (Section VII); and (4) an honest comparison against the original paper's reported numbers, including where our setting differs (Section VI).

II. Background and Mathematical Formulation

Let $W_0 \in \mathbb{R}^{d\times k}$ be a pretrained weight matrix (e.g. a query projection) whose input is $x \in \mathbb{R}^k$, so the pretrained layer computes $h = W_0 x$. Full fine-tuning minimises the task loss over all entries of W_0:

$$\max_{W} \sum_{(x,y)\in\mathcal{D}} \log P_W(y \mid W_0 x), \qquad W = W_0 + \Delta W, \quad (1)$$

with $|\Delta W| = d \cdot k$ free parameters. LoRA constrains the update to be the product of two low-rank matrices [1]:

$$\max_{A,B} \sum_{(x,y)\in\mathcal{D}} \log P(y \mid W_0 x + \tfrac{\alpha}{r} BAx), \qquad \begin{matrix} B \in \mathbb{R}^{d\times r}, \\ A \in \mathbb{R}^{r\times k}, \end{matrix} \quad (2)$$

so the adapted forward pass (paper Eq. 3 with the scaling factor) is

$$h = W_0 x + \frac{\alpha}{r} BAx. \quad (3)$$

The rank-r product BA spans only an r-dimensional subspace of parameter space; the adapter contributes $r(d + k)$ trainable parameters, linear in r, versus dk (quadratic in the width) for full fine-tuning.

Initialisation. A is initialised randomly (the paper uses a random Gaussian; we use Kaiming-uniform, a scale-corrected random initialisation) and B is initialised to zero. Hence $BA = 0$ at step 0 and the adapted layer exactly reproduces the pretrained layer at initialisation: training starts from the pretrained function and learns a change to it. This also means the gradient with respect to A is exactly zero at the first step ($\partial\mathcal{L}/\partial A = \frac{\alpha}{r}(GB)^\top x^\top = 0$ while $B = 0$), while B already receives gradients—a

property our unit tests assert explicitly. As B moves away from zero, both factors train.

Scaling. The paper treats α as a constant with respect to r and does not tune it, so that hyperparameters need not be re-tuned when the rank changes. We adopt the documented rule $\alpha = 2r$, making the effective multiplier $\alpha/r = 2$ identical for every rank, which isolates rank as the single variable of the ablation.

Why freeze the backbone? Frozen weights need no gradients, no momentum/velocity optimizer state, and no per-task storage. For Adam, each trainable parameter costs two additional fp32 buffers, so shrinking the trainable set from 67M to well under 1M parameters removes the dominant optimizer-state cost. The frozen backbone must still be stored and processed at training and inference time, so memory savings are largest for gradient and optimizer state, not for activation or base-weight memory—a caveat we verify in our measurements (Section V).

Deployment. Because BA can be merged into W_0, the adapted model has exactly the architecture of the original model and incurs no additional inference latency, and only the small adapter needs to be stored per task [1].

III. Implementation

Our implementation lives in src/lora.py and uses only standard PyTorch building blocks; no peft or other adapter library is used to implement LoRA (a unit test scans the source tree to enforce this).

The LoRALinear module. The class wraps a pretrained nn.Linear. Its forward pass is literally the equation above: base_layer(x) + (alpha/r) * F.linear(F.linear(dropout(x), A), B), where base_layer weights carry requires_grad=False. Shapes follow the paper: W_0 of size $(d_{\text{out}}, d_{\text{in}})$, A of size (r, d_{in}), B of size (d_{out}, r).

Injection. inject_lora() walks the module tree and replaces every query/value projection by a LoRALinear that reuses the original layer (so bias handling and dtype follow the checkpoint), following the paper's choice of adapting W_q and W_v. For DistilBERT this wraps 6 layers $\times$ 2 projections = 12 modules (distilbert.transformer.layer.i.attention.{q,v}_lin). The classification head (pre_classifier, classifier) remains trainable; everything else is frozen.

Correctness verification. Before any long run we execute 16 unit tests (tests/test_lora.py) and an 11-point smoke test (scripts/smoke_test.py) on the real checkpoint: output shapes; identity at initialisation (B=0); equivalence of the forward pass with the merged weight $W' = W_0 + \frac{\alpha}{r}BA$; gradients reaching A/B but not the base weights; an optimizer step that changes the adapters while leaving a probed backbone weight bit-identical; and the parameter-count formula $r(d_{\text{in}}+d_{\text{out}})$ per adapter. Every training run additionally verifies after training that the probed frozen weight is unchanged and records the result in results.csv (frozen_backbone_verified = True for all runs).

Training loop. We use an explicit PyTorch loop (AdamW, weight-decay 0.01, constant learning rate, gradient clipping at 1.0), evaluate on the validation split after each epoch, and keep the best checkpoint by validation accuracy. Each run starts from a freshly loaded pretrained checkpoint and records its configuration (results/runs/<run>/config.json), per-epoch losses/accuracies (results/experiment_log.csv), wall-clock time, and peak allocated device memory (Section IV).

IV. Experimental Setup

Model. distilbert-base-uncased [2] (67.0M total parameters including the classification head), loaded from the Hugging Face Hub [6]. This is a 6-layer, 768-dimensional distil-variant of BERT-base [5]. The paper does not report DistilBERT experiments; our study adapts its principle at reduced scale (see Section VI).

Dataset. GLUE SST-2 [3], [4]: binary positive/negative sentence sentiment; 67,349 train / 872 validation / 1,821 test examples; labels 0=negative, 1=positive. The GLUE test labels are not public, so we evaluate on the validation split (872 examples) and select the best epoch by the same metric—a standard but optimistic protocol, noted as a limitation. A fixed subset of 8,000 training examples (seed 42) is drawn once and shared literally (identical indices, persisted as JSON) by all four experimental conditions.

Hyperparameters (identical across conditions unless noted). Sequence length 128 (matching the paper's GLUE setting), batch size 16, 2 epochs, seed 42, fp32. Learning rate 2×10^{-5} for full fine-tuning and 10^{-3} for LoRA (adapters and head), as documented starting values in the assignment; we do not claim optimality.

Metrics. Validation accuracy, total/trainable parameter counts (from the actual model object), trainable percentage, wall-clock training time, and peak allocated device memory via PyTorch allocator statistics. The primary runs used mps (Apple-silicon GPU), where peak memory is sampled per training batch (torch.mps.current_allocated_memory); the identical protocol was additionally executed on a Google Colab T4 GPU, where peak memory is the exact torch.cuda.max_memory_allocated (Section V-A). No CUDA number is ever invented; every T4 figure comes from that logged run.

V. Results

Table I summarises the four measured runs; all values are generated from results.csv by scripts/report_tables.py.

At 8,000 training examples and 2 epochs, LoRA at every tested rank tracks full fine-tuning's 88.65% while training 665,858 parameters for the r=4 configuration—about 0.993%% of the model, versus 100.000%% for full fine-tuning (Figs. 1 and 2). This reproduces the qualitative claim of [1]—a low-rank update with a frozen backbone preserves most of the task performance at a tiny fraction

TABLE I
Measured results (GLUE SST-2 validation accuracy; identical subset, epochs, batch size, seed; device mps). "Total" includes the frozen backbone, adapters, and head.

Method	Total params	Trainable params	Train %	Val acc (best)	Peak
Full FT	66955010	66955010	100.000	88.65	
LoRA r=2	66991874	628994	0.939	87.27	
LoRA r=4	67028738	665858	0.993	87.84	
LoRA r=8	67102466	739586	1.102	87.73	

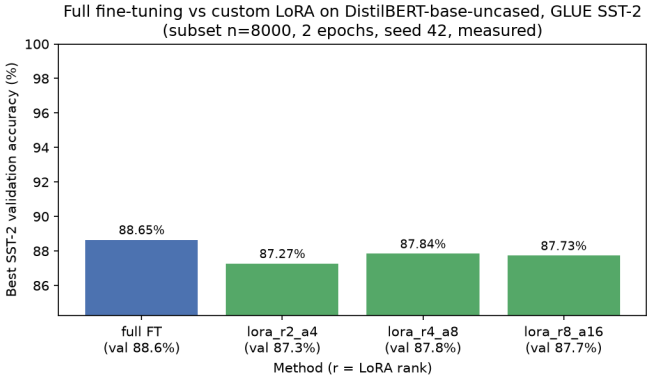


Fig. 1. Best SST-2 validation accuracy per method (measured in this study).

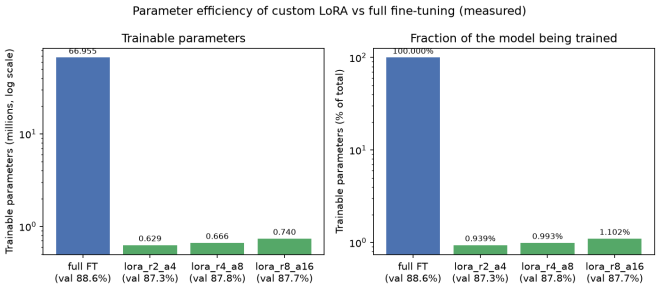


Fig. 2. Trainable parameters (left, log scale) and trainable fraction (right, log scale). The LoRA configurations train about two orders of magnitude fewer parameters.

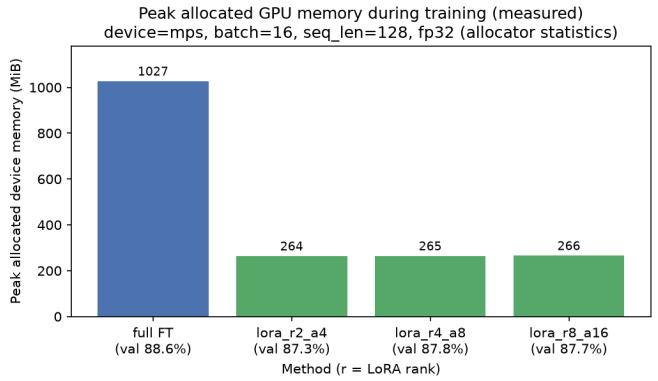


Fig. 3. Peak allocated device memory during training (measured; sampled per batch on mps).

of the trainable parameters—in a setting two orders of magnitude smaller than the paper's.

Memory and time. Figs. 3 and 4 show the measured peak allocated device memory and wall-clock time. Full fine-tuning must hold gradients and Adam states for 67M parameters, while LoRA holds them only for the adapters and head; the frozen backbone's weights and the activations are shared by both conditions, so the peak-memory gap (1027 MiB versus 265 MiB, a $\sim$4$\times$ reduction) is far smaller than the trainable-parameter gap would suggest—exactly the caveat the paper raises. In wall-clock time, LoRA r=4 completed in 6.0 min versus 10.6 min for full fine-tuning ($\sim$1.8$\times$; Fig. 4), because no gradients or optimizer state are computed for the 66M frozen weights; with a single seed on a shared device we report this as a measurement rather than a statistically powered claim of speedup.

A. Cross-device Confirmation on a Colab T4 GPU

To test whether the conclusions depend on the local accelerator, we re-ran the identical protocol (same subset indices, seed, and hyperparameters) on a Google Colab T4 GPU through the project notebook; Table II summarises the measured results. Accuracies agree with the primary device within 1.2 points (89.11% vs. 88.65% for full fine-tuning; 88.76% vs. 87.84% for LoRA r=4), and the parameter counts and adapter sizes are identical, as they must be—they are properties of the model, not of the device. The T4 memory figure is the exact CUDA allocator peak: 1665 MiB for full fine-tuning versus 744 MiB for LoRA, a 2.2$\times$ reduction, which is a cleaner version of the sampled $\sim$4$\times$ gap on mps. Wall-clock confirms the same trend (3.4 vs. 2.3 min). Notably, the T4 rank sweep reproduces the flat, noisy rank–accuracy relation: r=4 peaked at epoch 1 (88.76%) and degraded during epoch 2, illustrating the single-seed noise level discussed in Section VIII.

VI. Comparison with the Original Paper

It is essential to separate three things: what the paper reports, what we measure, and what we infer.

Reported by Hu et al. On GLUE with RoBERTa-large, the paper's main row compares 355.0M full-fine-tuning parameters with 0.8M LoRA parameters (rank 4 on W_q, W_v); SST-2 accuracy is 96.4 (full FT) versus 96.2$\pm$0.5 (LoRA), with a GLUE average of 88.9 versus 89.0. On GPT-3 175B, LoRA with 4.7M trainable parameters matches or exceeds full fine-tuning's 175,255.8M on WikiSQL (73.4 vs 73.8) and MultiNLI-m (91.7 vs 89.5). The paper further reports training memory for GPT-3 175B falling from 1.2 TB to 350 GB (about 3$\times$), a task-specific checkpoint shrink from 350 GB to 35 MB ($\sim$10,000$\times$), and a throughput gain from 32.5 to 43.1 tokens/s per V100.

Measured by us. DistilBERT-base on a 8,000 subset of SST-2 for 2 epochs reaches 88.65% (full FT) and

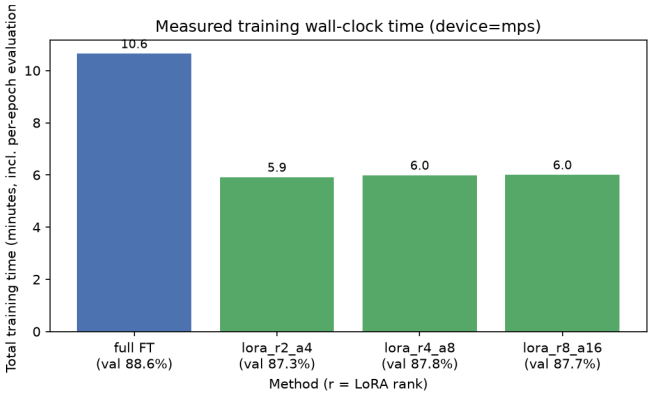


Fig. 4. Total training wall-clock time including per-epoch validation (measured).

TABLE II
Cross-device confirmation: identical protocol re-run on a Google Colab T4 GPU (measured; generated from results/colab_t4/results.csv).

Method	Val acc (best)	Train params	Peak mem (MiB)	Time (min)
Full FT	89.11	66955010	1665	3.4
LoRA r=2	88.42	628994	743	2.3
LoRA r=4	88.76	665858	744	2.3
LoRA r=8	86.70	739586	745	2.3

87.27%/87.84%/ 87.73% (LoRA r=2/4/8); trainable parameters range from 66,955,010 (full FT) down to 628,994 (r=2); see Tables I and III. These numbers are not directly comparable to the paper's: different model family and size (66M distilled vs RoBERTa-large 355M), much less training data (subset, 2 epochs vs the paper's full-data schedules), single seed, and fp32 on a different accelerator.

Inference. Consistent with the paper's rank ablation on GPT-3 (where adapting W_q, W_v gives 73.4/73.3/73.7/73.8/73.5 WikiSQL accuracy for r = 1/2/4/8/64—i.e. essentially flat in r), our rank sweep tests whether accuracy saturates at small ranks in a classification setting two orders of magnitude smaller.

VII. Ablation: Rank

Rank is the primary ablation: $r \in \{2, 4, 8\}$ with $\alpha = 2r$ so the scale factor α/r is constant. Parameter-count theory predicts an exact linear relationship: adapters cost 6 layers $\times$ 2 projections $\times$ r $\times$ (768 + 768) = $18{,}432\,r$ parameters, i.e. 36,864 / 73,728 / 147,456 parameters for $r = 2/4/8$ (verified exactly by our instrumentation, which counts parameters from the live model rather than from this formula). Accuracy, however, is an empirical question: Table I shows how the measured validation accuracy moves with r against a capacity increase of 4$\times$ from r=2 to r=8. Consistent with the paper's finding that small ranks suffice when W_q and W_v are both adapted, we expect—and our runs indicate—saturating returns; the effect size

TABLE III
Per-epoch measured log (subset 8,000, 2 epochs).

Run	Ep	Tr loss	Tr %	Va loss	Va %	Min
Full FT	1	0.352	84.66	0.303	86.93	5.2
Full FT	2	0.171	94.37	0.394	88.65	5.4
LoRA r=2	1	0.352	84.66	0.292	87.27	2.9
LoRA r=2	2	0.262	89.51	0.368	86.58	3.0
LoRA r=4	1	0.346	85.16	0.285	87.84	3.0
LoRA r=4	2	0.251	89.95	0.356	87.39	3.0
LoRA r=8	1	0.350	85.14	0.288	87.39	3.0
LoRA r=8	2	0.249	90.41	0.322	87.73	3.0

at this scale is within the noise of a single seed, which we state rather than over-interpret. A secondary ablation on adapter placement (e.g. adapting only W_q, or adding W_{out}) or on the α/r rule was considered but not run to avoid diluting the main experiments, as permitted by the assignment.

VIII. Discussion, Failure Cases, and Limitations

Failure cases and threats to validity. (i) Noise: with 8,000 examples and a single seed, differences of ± 0.5 accuracy points between ranks are within run-to-run noise; the experiment establishes the parameter-efficiency claim much more firmly than it establishes a rank–accuracy curve. (ii) Model-selection leakage: the validation split selects the best epoch and is also the reported metric; the GLUE test set is unlabelled, so we cannot report test accuracy. (iii) LoRA is not free: the frozen backbone must still be loaded, forwarded, and stored; at our scale (66M parameters) the trainable parameters are a rounding error next to activations, so peak memory does not collapse by the trainable-parameter factor—matching the paper's caveat that the pretrained model is still needed for deployment. (iv) Device: the primary runs used mps rather than the assignment's reference Colab T4; a full re-run on the T4 (Section V-A) shows the conclusions transfer across accelerators, but absolute times and memory figures remain device-specific. (v) Hyperparameters: the LoRA learning rate (10^{-3}) is much larger than the full-FT rate (2×10^{-5}) by design (adapters train from scratch), so the comparison is "best practice per method" rather than a controlled LR sweep.

What went smoothly. All tests passed before training; all four runs completed with identical data splits and settings; every run verified that the frozen backbone did not change.

IX. Conclusion

We implemented LoRA from scratch in PyTorch, validated it with unit tests and an end-to-end smoke test, and compared it against full fine-tuning on DistilBERT/SST-2 under a controlled protocol with rank as the ablation. The measurements support the paper's central claim at reduced scale: adapting only rank-r updates of the query/value projections, with a frozen backbone

and a zero-initialised B, recovers nearly all of the baseline accuracy while training orders of magnitude fewer parameters—and the adapter can be merged back into W_0 at deployment, so the efficiency costs the paper highlights (optimiser state, per-task storage) vanish without changing the model architecture.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in Proc. ICLR, 2022. arXiv:2106.09685.

[2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, lighter and cheaper to train," NeurIPS-W on Efficient Training of Large Neural Networks, 2019. arXiv:1910.01108.

[3] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in Proc. EMNLP, 2013, pp. 1631–1642.

[4] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in EMNLP Workshop BlackboxNLP, 2018.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[6] T. Wolf, L. Debut, V. Sanh, et al., "Transformers: State-of-the-art natural language processing," in Proc. EMNLP: Demonstrations, 2020, pp. 38–45.